Background Tasks

You can define background tasks to be run *after* returning a response.

This is useful for operations that need to happen after a request, but that the client doesn't really have to be waiting for the operation to complete before receiving the response.

This includes, for example:

- Email notifications sent after performing an action:
  - As connecting to an email server and sending an email tends to be "slow" (several seconds), you can return the response right away and send the email notification in the background.
- Processing data:
  - For example, let's say you receive a file that must go through a slow process, you can return a response of "Accepted" (HTTP `202`) and process the file in the background.

Using `BackgroundTasks`

First, import `BackgroundTasks` and define a parameter in your *path operation function* with a type declaration of `BackgroundTasks`:

Python 3.10+

```python
from fastapi import BackgroundTasks, FastAPI

app = FastAPI()


def write_notification(email: str, message=""):
    with open("log.txt", mode="w") as email_file:
        content = f"notification for {email}: {message}"
        email_file.write(content)


@app.post("/send-notification/{email}")
async def send_notification(email: str, background_tasks: BackgroundTasks):
    background_tasks.add_task(write_notification, email, message="some notification")
    return {"message": "Notification sent in the background"}
```

FastAPI will create the object of type `BackgroundTasks` for you and pass it as that parameter.

Create a task function

Create a function to be run as the background task.

It is just a standard function that can receive parameters.

It can be an `async def` or normal `def` function, **FastAPI** will know how to handle it correctly.

In this case, the task function will write to a file (simulating sending an email).

And as the write operation doesn't use `async` and `await`, we define the function with normal `def`:

Ask AI

Python 3.10+

```python
from fastapi import BackgroundTasks, FastAPI

app = FastAPI()


def write_notification(email: str, message=""):
    with open("log.txt", mode="w") as email_file:
        content = f"notification for {email}: {message}"
        email_file.write(content)


@app.post("/send-notification/{email}")
async def send_notification(email: str, background_tasks: BackgroundTasks):
    background_tasks.add_task(write_notification, email, message="some notification")
    return {"message": "Notification sent in the background"}
```

Add the background task

Inside of your *path operation function*, pass your task function to the *background tasks* object with the method `.add_task()`:

Python 3.10+

```python
from fastapi import BackgroundTasks, FastAPI

app = FastAPI()


def write_notification(email: str, message=""):
    with open("log.txt", mode="w") as email_file:
        content = f"notification for {email}: {message}"
        email_file.write(content)


@app.post("/send-notification/{email}")
async def send_notification(email: str, background_tasks: BackgroundTasks):
    background_tasks.add_task(write_notification, email, message="some notification")
    return {"message": "Notification sent in the background"}
```

`.add_task()` receives as arguments:

- A task function to be run in the background (`write_notification`).
- Any sequence of arguments that should be passed to the task function in order (`email`).
- Any keyword arguments that should be passed to the task function (`message="some notification"`).

Dependency Injection

Using `BackgroundTasks` also works with the dependency injection system, you can declare a parameter of type `BackgroundTasks` at multiple levels: in a *path operation function*, in a dependency (dependable), in a sub-dependency, etc.

FastAPI knows what to do in each case and how to reuse the same object, so that all the background tasks are merged together and are run in the background afterwards:

Python 3.10+

```python
from typing import Annotated
```

```python
from fastapi import BackgroundTasks, Depends, FastAPI

app = FastAPI()


def write_log(message: str):
    with open("log.txt", mode="a") as log:
        log.write(message)


def get_query(background_tasks: BackgroundTasks, q: str | None = None):
    if q:
        message = f"found query: {q}\n"
        background_tasks.add_task(write_log, message)
    return q


@app.post("/send-notification/{email}")
async def send_notification(
    email: str, background_tasks: BackgroundTasks, q: Annotated[str, Depends(get_query)]
):
    message = f"message to {email}\n"
    background_tasks.add_task(write_log, message)
    return {"message": "Message sent"}
```

🤓 **Other versions and variants**

Python 3.10+ - non-Annotated

Tip

Prefer to use the `Annotated` version if possible.

```python
from fastapi import BackgroundTasks, Depends, FastAPI

app = FastAPI()


def write_log(message: str):
    with open("log.txt", mode="a") as log:
        log.write(message)


def get_query(background_tasks: BackgroundTasks, q: str | None = None):
    if q:
        message = f"found query: {q}\n"
        background_tasks.add_task(write_log, message)
    return q


@app.post("/send-notification/{email}")
async def send_notification(
    email: str, background_tasks: BackgroundTasks, q: str = Depends(get_query)
):
    message = f"message to {email}\n"
    background_tasks.add_task(write_log, message)
    return {"message": "Message sent"}
```

In this example, the messages will be written to the `log.txt` file *after* the response is sent.

If there was a query in the request, it will be written to the log in a background task.

And then another background task generated at the *path operation function* will write a message using the `email` path parameter.

Technical Details

The class `BackgroundTasks` comes directly from `starlette.background`.

It is imported/included directly into FastAPI so that you can import it from `fastapi` and avoid accidentally importing the alternative `BackgroundTask` (without the `s` at the end) from `starlette.background`.

By only using `BackgroundTasks` (and not `BackgroundTask`), it's then possible to use it as a *path operation function* parameter and have **FastAPI** handle the rest for you, just like when using the `Request` object directly.

It's still possible to use `BackgroundTask` alone in FastAPI, but you have to create the object in your code and return a Starlette `Response` including it.

You can see more details in Starlette's official docs for Background Tasks.

Caveat

If you need to perform heavy background computation and you don't necessarily need it to be run by the same process (for example, you don't need to share memory, variables, etc), you might benefit from using other bigger tools like Celery.

They tend to require more complex configurations, a message/job queue manager, like RabbitMQ or Redis, but they allow you to run background tasks in multiple processes, and especially, in multiple servers.

But if you need to access variables and objects from the same **FastAPI** app, or you need to perform small background tasks (like sending an email notification), you can simply just use `BackgroundTasks`.

Recap

Import and use `BackgroundTasks` with parameters in *path operation functions* and dependencies to add background tasks.